

STATEMENTS, EXPRESSIONS, AND OPERATORS

Outline



- Statement
- Expression
- C's assignment, mathematical, relational, and logical operators
- Operator precedence

Statement

- A **statement** is a complete direction instructing the computer to carry out some task.
 - ▣ usually written one per line
 - ▣ usually end with semicolon

Examples:

```
x = 2 + 3;
```

```
printf("Hello, World!");
```

Statements and White Space

- **Whitespace** refers to **tabs, spaces, and blank lines** in the code.
- Consider the following:

$X=2+3;$

$X = 2 + 3;$

X **$=$**
 2
 $+$
 $3;$

Are these valid
statements?

YES

Are they
equivalent?

YES

Statement and White Space

- What do the examples indicate?
 - ▣ C doesn't care about white space.
- For C statements and expressions, whitespace has no significant effect to how the program works.

Statement and White Space

- Now, consider this example:

```
printf("Hello,  
World!");
```

Is this legal?

NO

- The rule that C doesn't care about white space has one exception. **Within string literal constants, tabs and spaces aren't ignored – they are part of the string.**

Statement and White Space

- With that exception, consider this example.

```
printf("Hello, World!"  
);
```

Is this
legal?

YES

- Are the following equivalent?

```
printf("    Hello, World!");
```

```
printf("Hello,    W o r l d    !");
```

```
printf(" Hello,    World    !");
```

Null (or Empty) Statement

- A **null (or empty) statement** is placing a semicolon by itself on a line.
- This statement contains no instruction.

Compound Statement

- A **compound statement** is a group of two or more C statements enclosed in braces.
- A compound statement is also called a ***block***.
- Examples:

```
{  
    printf("Hello, ");  
    printf("World!");  
}
```

```
{ printf("Hello, ");  
  printf("World!"); }
```

Expression

- In C, an **expression** is anything that evaluates to a numeric value.
- Some simple expressions:

<i>Expression</i>	<i>Description</i>
PI	A symbolic constant (defined in the program)
20	A literal constant
rate	A variable
-1.25	Another literal constant

Expression

- An expression can take a complex form.

$1.25 * 8 + 5 * \text{RATE} / 2 * \text{COST}$

- The following statements are possible in a program:

$y = x = a + 10;$

$x = 6 + (y = 4 + 5);$

Operator

- An **operator** is a symbol that instructs C to perform some operation, or action, on one or more operands.
- An ***operand*** is something that an **operator acts on**.
- The categories of operators: **assignment, mathematical, relational, and logical**.

Assignment Operator

- In C, the assignment operator is represented by the **equal sign (=)**.
- The use of the equal sign (=) in programming is somewhat different in regular math.
- What does this statement mean?

$x = y;$

Assignment Operator

- C's assignment statement takes the form

variable = expression;

Examples:

x = 1;

z = a + c * 4 % 2;

Mathematical Operators



- In the C language, there are
two unary mathematical operators, and
five binary mathematical operators

Unary Mathematical Operators

- A **unary operator** acts on a single operand.
- The following are C's unary operators:

Operator	Symbol	Action	Examples
Increment	++	Increments the operand by one	++x, x++
Decrement	--	Decrements the operand by one	--x, x--

Unary Mathematical Operators

$++X;$

is equivalent to
 $X = X + 1;$

$--X;$

is equivalent to
 $X = X - 1;$

- C unary operator can be placed either before its operand (***prefix mode***) or after its operand (***postfix mode***).

$X++$

$++X$

$X--$

$--X$

Unary Mathematical Operators

- Consider these statements,

$x = 10;$

$y = x++;$

- After execution of the statements, what would be the value of x and y ?

x is 11

y is 10

Unary Mathematical Operators

- Consider these statements,

x = 10;

y = ++x;

- After execution of the statements, what would be the value of x and y?

x is 11

y is 11

Unary Mathematical Operators

- Changing a variable multiple times in the same expression produces an undefined result
- Consider this:

`x++ + x++`

The result is undefined. C may execute both `x++` operations together, the left `x++` first, or the right `x++` first

Solution? Avoid such expression!

Unary Mathematical Operators

```
1:  /* Demonstrates unary operator prefix and postfix modes */
2:
3:  #include <stdio.h>
4:
5:  int a, b;
6:
7:  main()
8:  {
9:      /* Set a and b both equal to 5 */
10:
11:      a = b = 5;
12:
13:      /* Print them, decrementing each time. */
14:      /* Use prefix mode for b, postfix mode for a */
15:
16:      printf("\n%d    %d", a--, --b);
17:      printf("\n%d    %d", a--, --b);
18:      printf("\n%d    %d", a--, --b);
19:      printf("\n%d    %d", a--, --b);
20:      printf("\n%d    %d\n", a--, --b);
21:
22:      return 0;
23: }
```

5	4
4	3
3	2
2	1
1	0

Binary Mathematical Operators

- A **binary operator** acts on two operands.
- The following are C's binary operators.

Operator	Symbol	Action	Example
Addition	+	Adds two operands	$x + y$
Subtraction	-	Subtracts the second operand from the first operand	$x - y$
Multiplication	*	Multiplies two operands	$x * y$
Division	/	Divides the first operand by the second operand	x / y
Modulus	%	Gives the remainder when the first operand is divided by the second operand	$x \% y$

Binary Mathematical Operators

- You already know the first four operators – addition, subtraction, multiplication, division.
- The fifth operator, modulus, is of interest here.
- It gives the remainder when the first operand is divided by the second operand.

$$100 \% 9 = ?$$

$$40 \% 6 = ?$$

$$10 \% 5 = ?$$

$$5 \% 3 = ?$$

Operator Precedence and Parentheses

- In an expression that contains more than one operator, what is the order in which operations are performed?

$X = 4 + 5 * 3;$

- The following shows C's mathematical operators precedence

Operators	Relative Precedence
++ --	1
* / %	2
+ -	3

Operator Precedence and Parentheses

- In expression with more than one operator,
 - ▣ if same precedence level, they are performed in left-to-right order as they appear in the expression
 - ▣ if different precedence levels, operators with higher precedence levels are performed first

$$12 \% 5 * 2 = ?$$

$$12 + 2 - 3 * 4 = ?$$

Operator Precedence and Parentheses

- To what value does this expression evaluate?

$x = 4 + 5 * 3;$

- What if you wanted to add 4 to 5 and then multiply the sum by 3?

$x = (4 + 5) * 3;$

Operator Precedence and Parentheses: Expression Evaluation

□ Example:

$$\begin{aligned}x &= 25 - (2 * (10 + (8 / 2))); \\ &= 25 - (2 * (10 + 4)) \\ &= 25 - (2 * 14) \\ &= 25 - 28 \\ &= -3\end{aligned}$$

DO use parentheses to make the order of expression evaluation clear.

DON'T overload an expression with parentheses.

Relational Operators

- C's relational operators are used to compare expressions.
- The following are C's relational operators.

Operator	Symbol	Question Asked	Example
Equal	==	Is operand 1 equal to operand 2?	x == y
Greater than	>	Is operand 1 greater than operand 2?	x > y
Less than	<	Is operand 1 less than operand 2?	x < y
Greater than or equal to	>=	Is operand 1 greater than or equal to operand 2?	x >= y
Less than or equal to	<=	Is operand 1 less than or equal to operand 2?	x <= y
Not equal	!=	Is operand 1 not equal to operand 2?	x != y

Relational Operators

- The following shows the usage of the relational operators

Expression	How It Reads	What It Evaluates To
$5 == 1$	Is 5 equal to 1?	0 (false)
$5 > 1$	Is 5 greater than 1?	1 (true)
$5 != 1$	Is 5 not equal to 1?	1 (true)
$(5 + 10) == (3 * 5)$	Is (5 + 10) equal to (3 * 5)?	1 (true)

Evaluating Relational Expressions

- Relational expressions evaluate to a value of either true (1) or false (0).

```
1:  /* Demonstrates the evaluation of relational expressions */
2:
3:  #include <stdio.h>
4:
5:  int a;
6:
7:  main()
8:  {
9:      a = (5 == 5);          /* Evaluates to 1 */
10:     printf("\na = (5 == 5)\na = %d", a);
11:
12:     a = (5 != 5);          /* Evaluates to 0 */
13:     printf("\na = (5 != 5)\na = %d", a);
14:
15:     a = (12 == 12) + (5 != 1); /* Evaluates to 1 + 1 */
16:     printf("\na = (12 == 12) + (5 != 1)\na = %d\n", a);
17:     return 0;
18: }
```

Precedence of Relational Operators

- All relational operators have lower precedence than the mathematical operators.

$$x+2>y$$

is the same as

$$(x+2)>y$$

Precedence of Relational Operators

- The following shows the order of precedence of C's relational operators.

Operators	Relative Precedence
< <= > >=	1
!= ==	2

x == y > z

Logical Operators

- Sometimes, it is necessary to ask more than one relational operator at once.
 - ▣ For example: **“if it is 7:00am AND it is a weekday AND it is not my vacation, ring the alarm”**
- The following shows C’s logical operators.

Operator	Symbol	Example
AND	&&	<i>exp1 && exp2</i>
OR		<i>exp1 exp2</i>
NOT	!	<i>!exp1</i>

Logical Operators

□ Usage of C's logical operators

Expression	What It Evaluates To
$(exp1 \ \&\& \ exp2)$	True (1) only if both $exp1$ and $exp2$ are true; false (0) otherwise
$(exp1 \ \ exp2)$	True (1) if either $exp1$ or $exp2$ is true; false (0) only if both are false
$(!exp1)$	False (0) if $exp1$ is true; true (1) if $exp1$ is false

Expression	What It Evaluates To
$(5 == 5) \ \&\& \ (6 != 2)$	True (1), because both operands are true
$(5 > 1) \ \ (6 < 1)$	True (1), because one operand is true
$(2 == 1) \ \&\& \ (5 == 5)$	False (0), because one operand is false
$!(5 == 4)$	True (1), because the operand is false

Logical Operators

Examples

- How would you write “is x equal to 2, 3, or 4?” in a code?
- How would you write “is n between x and y?” in a code?
- How would you write “is x outside the range 5 to 90?” in a code?

More on AND (&&) and OR (||)

Expression and its evaluation

□ ***exp1* && *exp2***

- ▣ If ***exp1*** is false, the whole expression evaluates to false regardless of the result of ***exp2***. In this case, ***exp2*** will no longer be evaluated.
- ▣ If ***exp1*** is true, the whole expression will evaluate to: true if ***exp2*** is true, or false if ***exp2*** is false. In this case, ***exp2*** will have to be evaluated before the result of the whole expression can be determined.

More on AND (&&) and OR (||)

Expression and its evaluation

□ ***exp1* || *exp2***

- ▣ If ***exp1*** is true, the whole expression evaluates to true regardless of the result of ***exp2***. In this case, ***exp2*** will no longer be evaluated.
- ▣ If ***exp1*** is false, the whole expression will evaluate to: true if ***exp2*** is true, or false if ***exp2*** is false. In this case, ***exp2*** will have to be evaluated before the result of the whole expression can be determined.

More on True/False values

- A **value of zero** represents **false**.
- Any **nonzero value** represents **true**.
- Example:

```
x = 13;
```

```
z = x && 1;
```

```
y = x || 0;
```

Precedence of Operators

- C's logical operators have a precedence order – both among themselves and in relation to other operators.

Precedence of Operators

- Consider this: you want to write a logical expression that makes these comparisons,
 1. Is a less than b?
 2. Is a less than c?
 3. Is c less than d?
- And you want the entire logical expression to evaluate to true if condition 3 is true AND if either condition 1 OR condition 2 is true.

Precedence of Operators

□ Which of the following is correct for what you wanted to do?

a) **$a < b \mid \mid a < c \&\& c < d$**

b) **$a < b \mid \mid (a < c \&\& c < d)$**

c) **$(a < b \mid \mid a < c) \&\& c < d$**

Precedence of Operators

```
1:  #include <stdio.h>
2:
3:  /* Initialize variables. Note that c is not less than d, */
4:  /* which is one of the conditions to test for. */
5:  /* Therefore, the entire expression should evaluate as false.*/
6:
7:  int a = 5, b = 6, c = 5, d = 1;
8:  int x;
9:
10: main()
11: {
12:     /* Evaluate the expression without parentheses */
13:
14:     x = a < b || a < c && c < d;
15:     printf("\nWithout parentheses the expression evaluates as %d", x);
16:
17:     /* Evaluate the expression with parentheses */
18:
19:     x = (a < b || a < c) && c < d;
20:     printf("\nWith parentheses the expression evaluates as %d\n", x);
21:     return 0;
22: }
```

Compound Assignment Operators

- This provides a shorthand method for combining a binary mathematical operation with an assignment operation.

exp1 = exp1 op exp2; → x = x + 5;

exp1 op= exp2; → x += 5;

Compound Assignment Operators

- The following shows some compound assignment operators

When You Write This...	It Is Equivalent To This
$x *= y$	$x = x * y$
$y -= z + 1$	$y = y - z + 1$
$a /= b$	$a = a / b$
$x += y / 8$	$x = x + y / 8$
$y \% = 3$	$y = y \% 3$

Operator Precedence and Associativity

Level	Operator	Operation	Associativity
<i>Highest</i> <i>Lowest</i>	++, --	Postfix increment, Postfix decrement	left to right
	++, --, +, -	Prefix increment, Prefix decrement, Unary plus, Unary minus	right to left
	*, /, %	Multiplication, Division, Modulus	left to right
	+, -	Addition, Subtraction	left to right
	<, <=, >, >=	Relational: less than, less than or equal to, greater than, greater than or equal to	left to right
	==, !=	Relational: equal to, not equal to	left to right
	&&	Logical AND	left to right
		Logical OR	left to right
	=, +=, -=, *=, /=, %=	Assignment, Compound Assignments	right to left

Operator Precedence

Level	Operators
1	() [] -> .
2	! ~ ++ -- * (<i>indirection</i>) & (<i>address-of</i>) (<i>type</i>) sizeof + (<i>unary</i>) - (<i>unary</i>)
3	* (<i>multiplication</i>) / %
4	+ -
5	<< >>
6	< <= > >=
7	== !=
8	& (<i>bitwise AND</i>)
9	^
10	
11	&&
12	
13	?:
14	= += -= *= /= %= &= ^= = <<= >>=
15	,
() is the function operator; [] is the array operator.	